COMPUTER SCIENCE + COMPUTER ENGINEERING

Module 20 Engineering practice, security and maintainability

Individual module / Expanded Volume 04

Four lessons with original foundations, expanded study, worked examples, laboratory checkpoints and quizzes.

Lessons 20.1–20.4 · 20 questions · One integration project

Package 05 · Expanded Volume 04 · September 28, 2026
Self-study teaching edition. Completion requires independent implementation, testing and review.

Module 20: Engineering practice, security and maintainability

Original prerequisites: Volumes 01-04.

Integration goal: Gate D evidence and release. Assemble the earlier investigations into the local workload-processing application. Complete the Gate D specification rather than adding a public deployment.

20.1 — Requirements, architecture and design records

Learning objective: Make tradeoffs explicit and trace implementation back to needs.

Original foundation

A requirement states an observable need or constraint. Functional requirements describe behavior; quality requirements describe properties such as latency, reliability or accessibility. Architecture allocates responsibilities and dependencies to meet those requirements. A diagram without contracts and failure behavior is not enough. An architecture decision record states the problem, constraints, alternatives, chosen approach and consequences. Avoid premature distributed systems or custom frameworks when a smaller design meets the need. Revisit decisions when evidence changes. Traceability links requirements to tests and avoids declaring success merely because all planned code was written.

Expanded study

A requirement needs an observation boundary

"Fast" and "reliable" do not identify what to measure. A useful requirement describes the workload, environment, result and acceptance rule. For ForgeQueue, an example is: reject an advertised body length above 4096 before allocating that body. Another is: with capacity four, no more than four accepted jobs may be queued or running inside the service. State separately whether completed results retained by a caller are inside that budget.

Map responsibilities before distributing them

The reference separates framing, request validation, admission, native computation and persistence. Python handles the local orchestration; a trusted C++ executable performs the bounded arithmetic. The small workload deliberately makes overhead visible. It does not establish that launching a process is a good per-frame engine architecture. The C++ queue and task-graph labs teach in-process concurrency separately. Choosing a boundary means accepting its copying, lifetime and failure costs.

Write a decision that can be revisited

An architecture decision record includes context, constraints, alternatives, the selected option and consequences. For example, a SQLite result store avoids requiring a server for this local learning project, but it narrows the concurrency and deployment assumptions. A no-network file-driven interface may be even simpler; a loopback gateway is useful when the learning objective includes framing and timeouts. A rejected alternative is not universally bad; it was not selected under these constraints.

Trace requirements to evidence

The requirements sheet names the implementation boundary and an acceptance test for each claim. A test name is not a test result, and a green pipeline cannot establish requirements it never exercised. Include reviewer judgment for ownership arguments, threat assumptions and accessibility observations. Keep the source program's Gate D requirements intact while marking the new ForgeQueue design as one worked realization, not the only permissible student project.

Worked reasoning

A complete requirement might read: "REQ-3: On a retry with the same validated ID and identical payload against the same retained database history, return the committed response without inserting a second result row." Acceptance evidence includes a direct repeat, concurrent repeats, a restart and a conflict case. Merely checking that two calls returned HTTP-like success text would not establish the durable-row invariant.

Runnable example

File: `python/examples/lesson_20_1.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 20.1 --build-dir build
```

```
requirements={"REQ-1":"reject oversized frame","REQ-2":"drain accepted work","REQ-3":"deduplicate
durable result"}
evidence={"REQ-1":["test_oversize"],"REQ-2":[],"REQ-3":["test_replay"]}
missing=[r for r in requirements if not evidence.get(r)]
print("Requirements missing evidence:",missing)
print("Named tests still need to be run and reviewed; names are not proof.")
```

Expected output from the supplied reference in the validated environment:

```
Requirements missing evidence: ['REQ-2']
Named tests still need to be run and reviewed; names are not proof.
```

Practical checkpoint

Write six measurable requirements and two decision records for the simulation. Pass: every requirement has an acceptance test and each decision includes a rejected alternative and a cost.

Expanded contract: Evidence lab: requirements, traceability and two architecture decision records. The demonstration only detects missing evidence entries; it does not certify architecture quality or execute named tests.

Complete the four implementation/evidence steps and the additional problems in the workbook. This is an evidence-based lab; a function-only checker cannot assess the required review and documentation.

Independent transfer

Every planned programming task is complete, but the service accepts unbounded jobs. Is Gate D complete?

Quiz — answer before opening the key

1. **What makes a requirement testable?**
2. **Why record rejected alternatives?**
3. **Is completing planned tasks the same as satisfying requirements?**
4. **Why name the retained database history in a replay requirement?**

5. **What evidence is missing when a requirements matrix contains only implementation filenames?**

Assigned study

R01: Software Engineering; R02: engineering design and professional practice.

Expansion source map: R01, R02, R05. See the source directory for the exact documents and version notes.

20.2 — Defensive programming and threat modeling

Learning objective: Protect assets and trust boundaries before selecting security mechanisms.

Original foundation

A threat model identifies assets, actors, entry points, trust boundaries and plausible failures. Game clients, mods, save files and asset importers are not automatically trusted. Validate input, minimize privileges and isolate risky processing. Memory-safe interfaces, bounds checks, fuzzing and sanitizers help address different defect classes. Authentication establishes an identity or credential; authorization decides permitted actions. Encryption alone does not enforce game rules or protect a compromised endpoint. Use established security libraries rather than inventing cryptography. Perform testing only against your own code or authorized lab targets; the purpose here is defensive engineering.

Expanded study

Identify assets and authority

The reference protects workstation availability, private scratch files and the meaning of stored results. Entry points include request bytes, configuration paths and the configured native executable. A client supplies bounded data, not executable code. The parent directory and native binary are trusted owner configuration. The loopback binding reduces unintended exposure but does not authenticate local users, and the project must not be described as a public-ready server.

Bound more than one resource

A payload length limit does not bound the number of accepted jobs. A worker-count limit does not necessarily bound an executor's waiting queue. ForgeQueue uses a semaphore for queued-plus-running work, fixed payload/value limits, a subprocess wait budget and a durable-history quota. The code assumes the shipped trusted worker emits bounded output; it is not a sandbox for an arbitrary hostile executable that floods output or consumes unlimited memory. Record that residual risk explicitly.

Fuzz within a declared target

The demonstration feeds bounded generated byte strings only to the supplied parser. Random bytes mostly exercise early rejection, so a stronger lab also mutates valid messages: remove fields, duplicate fields, change types, cross length boundaries and preserve enough structure to reach later validation. Keep the seed and the smallest failing input. A fixed input count with no unexpected exception is useful observed evidence, not proof that the parser is secure. [P04, P05]

Give failures useful but restrained messages

An operator needs to distinguish invalid input, overload, worker failure and uncertain outcome. Full request bodies, filesystem paths, credentials and internal stack traces usually do not belong in a client-facing message. Keep detailed diagnostics in a controlled local record when needed. Encryption and authentication would be separate additions for a different deployment; they would not eliminate validation or authorization responsibilities.

Worked reasoning

A four-byte header advertises 4,294,967,295 bytes. The receiver rejects it before requesting a body of that size. A correctly sized request asks for operation "exec"; the parser rejects it because only sum_squares is allowed. A valid request arrives while all four admission tickets are occupied; the service returns BUSY. These failures occur at different boundaries and should not all become one misleading "bad JSON" message.

Runnable example

File: `python/examples/lesson_20_2.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 20.2 --build-dir build
```

```
import random
from python.reference.lab_19_2 import parse_request
rng=random.Random(2026);rejected=0
for _ in range(500):
    raw=bytes(rng.randrange(256) for _ in range(rng.randrange(129)))
    try:
        parse_request(raw)
    except ValueError:
        rejected+=1
print("bounded random inputs=500 rejected=",rejected,sep="")
print("No observed unexpected exception; not a security proof.")
```

Expected output from the supplied reference in the validated environment:

```
bounded random inputs=500 rejected=500
No observed unexpected exception; not a security proof.
```

Practical checkpoint

Threat-model the save parser and local service. Add bounds checks and a small malformed-input fuzz harness. Pass: cap work/memory, preserve useful errors and record residual risks instead of claiming complete security.

Expanded contract: Evidence lab: threat model plus bounded fuzz/mutation report against this package only. No network scanning, public deployment or untrusted executable execution is required. Claims must distinguish tests, design arguments and untested assumptions.

Complete the four implementation/evidence steps and the additional problems in the workbook. This is an evidence-based lab; a function-only checker cannot assess the required review and documentation.

Independent transfer

The service binds only to 127.0.0.1. Does that authorize every local caller?

Quiz — answer before opening the key

1. **How does authorization differ from authentication?**
2. **Why is a game client a trust boundary?**
3. **Does encryption replace input validation?**
4. **Why does a worker-count limit alone not guarantee bounded backlog?**
5. **What additional fuzz cases reach beyond random-byte rejection?**

Assigned study

R01: Security; R11: bounds/lifetime safety; R02: information-security breadth.

Expansion source map: R01, R02, R11, P04, P05, P09, P14. See the source directory for the exact documents and version notes.

20.3 — Build automation, dependencies and release discipline

Learning objective: Produce a verifiable artifact from a known source revision.

Original foundation

Continuous integration runs repeatable checks when changes are made. A useful pipeline builds, tests and reports failures from a clean environment. Pin dependencies sufficiently to reproduce the result and record toolchain versions. A dependency inventory includes origin, version and license obligations; it is not enough to know that a package downloaded successfully. Separate secrets from source and limit automation permissions. Preserve symbols or diagnostics needed to investigate crashes, while controlling access to sensitive information. A release artifact should be traceable to a commit and build configuration. Automated success is evidence only for the checks actually run.

Expanded study

Define what is reproduced

A clean build should produce a runnable artifact from a known source tree and documented toolchain. That is different from a claim that two builds are bit-for-bit identical. Compiler versions, generated metadata and archive timestamps can affect bytes. The package supplies a source/file manifest and exact verification commands; they identify tested inputs without promising identical binaries across compilers or operating systems. [R05, P13]

Fail before publishing

The local verification script configures the native project, builds it, runs CTest, runs the Python tests and executes the demonstrations. A nonzero result must stop the process. The release helper invokes verification before creating a new archive; its explicit failure-injection option checks that a failed step does not generate a fresh release. Do not suppress errors to make a release button look successful.

Inventory runtime and build dependencies

The code requires Python, its sqlite3 module, a C++20-capable toolchain, CMake and platform thread support. No package download happens during the verification script. Record concrete versions from the actual machine and distinguish them from the document-rendering tools used to author the PDFs. External reading links are not bundled third-party source. A dependency inventory should note provenance and applicable terms rather than inventing legal conclusions.

Treat integrity and authenticity separately

SHA-256 lets a reviewer compare a file with an expected digest. If both the file and its digest are replaced by an untrusted party, that comparison alone establishes no trusted origin. The manifest excludes its own digest to avoid a self-reference loop and states the exclusion. Preserve diagnostic output needed for a failure, but do not accidentally archive secrets, local databases or unrelated home-directory files.

Worked reasoning

Change one byte in native/worker.cpp: its digest changes. That detects a mismatch against the original recorded value. Now intentionally force the verification process to return nonzero. The release helper must exit without creating the requested new ZIP. Finally perform a clean successful run, identify the source tree and toolchain, and extract the release into another directory to rerun the same documented checks.

Runnable example

File: `python/examples/lesson_20_3.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 20.3 --build-dir build
```

```
import hashlib
source=ROOT/"native"/"worker.cpp"
before=hashlib.sha256(source.read_bytes()).hexdigest()
changed=hashlib.sha256(source.read_bytes()+b"\n").hexdigest()
print("change detected=",before!=changed,sep="")
print("A digest detects byte changes against a trusted value; it is not authentication.")
```

Expected output from the supplied reference in the validated environment:

```
change detected=True
A digest detects byte changes against a trusted value; it is not authentication.
```

Practical checkpoint

Create a local clean-build script that runs tests and produces a versioned archive plus manifest. Pass: run it from a fresh build directory and deliberately break a test to confirm the pipeline stops.

Expanded contract: Evidence lab: clean build/test log, failed-pipeline demonstration, dependency inventory, versioned source release and extraction/rebuild evidence. A hash manifest is an integrity aid, not an authenticity certificate.

Complete the four implementation/evidence steps and the additional problems in the workbook. This is an evidence-based lab; a function-only checker cannot assess the required review and documentation.

Independent transfer

The pipeline printed a failed test but returned exit code zero and created a release. Does it satisfy the acceptance rule?

Quiz — answer before opening the key

1. **What is the purpose of dependency pinning?**
2. **What should identify a release artifact's origin?**
3. **Why test a pipeline with an intentional failure?**
4. **Why does the package manifest exclude its own hash?**
5. **Does the same source hash guarantee identical executable bytes on two compilers?**

Assigned study

R05: workflows and tagging; R12: builds and testing; R01: software engineering.

Expansion source map: R05, R12, P13. See the source directory for the exact documents and version notes.

20.4 — Ethics, accessibility, teamwork and sustainable delivery

Learning objective: Evaluate engineering decisions beyond code correctness.

Original foundation

Engineering work affects players, colleagues and the public. Accessibility, privacy, intellectual property, honest reporting and sustainable maintenance belong in technical decisions. Obtain permission for assets and code and respect the licenses under which they are supplied. Do not assume publicly visible material is free to redistribute. Teams need clear responsibilities, respectful review and realistic scope. Crunch is not a substitute for planning and can hide systemic delivery problems. A postmortem should identify contributing conditions and improvements, not scapegoats. For hardware, safety and environmental constraints become additional design requirements, not optional polish.

Expanded study

Evaluate the operator experience as part of correctness

A service may calculate the right result and still be difficult to use when its status messages are ambiguous or its recovery instructions are missing. The reference uses textual status and stable error codes rather than color alone. A learner should test small-window/terminal use, keyboard-only operation where relevant and whether a second person can recover from an ordinary input mistake. These are review activities, not outcomes that a unit test suite can fully certify. [R29]

Keep privacy and attribution concrete

Use generated or consented data for the course. A bug report usually does not need a player's real name, credentials or an entire private save file when a small synthetic reproduction demonstrates the defect. Record which code is original student work, which is supplied reference material and which external components are used. Public access to a resource is not a reason to omit its provenance or applicable permissions. [R01, R02]

A retrospective explains contributing conditions

An incident review should name the observed failure, its impact, detection, timeline, contributing design or process conditions, and specific improvements. "The developer was careless" does not describe a reproducible mechanism or prevention check. A better explanation might be that cancellation tests never included a queued job and the release process did not exercise shutdown under load. That finding suggests an owner, a new test and an acceptance rule.

Finish at an honest scope

Gate D is a small defensible systems application, not a commercial online backend. Preserve unresolved assumptions and platform limits in the handoff. A smaller working scope with clear evidence is preferable to adding public hosting, arbitrary plugins or extra protocols without corresponding tests and review. Passing the code checks does not replace a live explanation, independent extension or the learner's own failure investigation.

Worked reasoning

Replace "ERROR: invalid" with "REQ_VERSION: expected version 1. Check the request template and retry." The new message supplies a stable category and an action without echoing a full untrusted payload. For an uncertain post-submission timeout, a different message must say to retry the same ID and payload because work may already have committed. Reusing the invalid-input wording would give the operator the wrong recovery model.

Runnable example

File: `python/examples/lesson_20_4.py`. This is a worked reference example, not the student exercise.

```
python3 tools/run_example.py 20.4 --build-dir build
```

```
old="ERROR: invalid"
new="REQ_VERSION: expected version 1. Check the request template and retry."
print("Before:",old)
print("After:",new)
print("Review: text-only status, actionable recovery, no secret or full payload in the message.")
```

Expected output from the supplied reference in the validated environment:

```
Before: ERROR: invalid
After: REQ_VERSION: expected version 1. Check the request template and retry.
Review: text-only status, actionable recovery, no secret or full payload in the message.
```

Practical checkpoint

Run an accessibility review and a blameless failure retrospective on the project. Pass: identify at least three actionable changes, assign acceptance evidence and document third-party attribution honestly.

Expanded contract: Evidence lab: usability/accessibility review, blameless incident report, attribution/dependency record and Gate D handoff. Do not mark human review complete because an automated example printed a proposed message.

Complete the four implementation/evidence steps and the additional problems in the workbook. This is an evidence-based lab; a function-only checker cannot assess the required review and documentation.

Independent transfer

An optimization removes clear errors and silently retries forever. Is higher throughput sufficient justification?

Quiz — answer before opening the key

1. **Does public availability imply permission to redistribute?**
2. **Why include accessibility in acceptance criteria?**
3. **What should a blameless retrospective seek?**
4. **Why must UNKNOWN_OUTCOME have different recovery instructions from INVALID_REQUEST?**
5. **What makes a retrospective action testable?**

Assigned study

R29: input, text, captions and UI guidance; R01/R02: professional and ethical practice.

Expansion source map: R01, R02, R29. See the source directory for the exact documents and version notes.

Module integration checkpoint

Original project brief: Complete Gate D: a reviewed systems application with a design record, threat model, CI-style local test pipeline, dependency inventory and an incident/recovery exercise.

Assemble the earlier investigations into the local workload-processing application. Complete the Gate D specification rather than adding a public deployment.

1. Six measurable requirements, two reviewed decision records and test-to-requirement traceability.
2. A thread/resource ownership map, protocol/schema and explicit commit/cancel boundary.
3. A bounded threat model and malformed-input harness; residual risks and diagnostic limits.
4. A fresh build, versioned source archive and manifest; intentionally failed tests prevent release.
5. An operator/accessibility review, blameless recovery retrospective, dependency inventory and independent handoff.

Live review: Give a reviewer a fresh extraction. Reproduce one recovery case, explain one measured bottleneck and implement a small changed requirement without copying a solution.

The workbook supplies the additional exercises and five-question module review. Preserve at least five new edge/failure cases beyond copied examples. A mentor reviews the reasoning as well as the outputs.